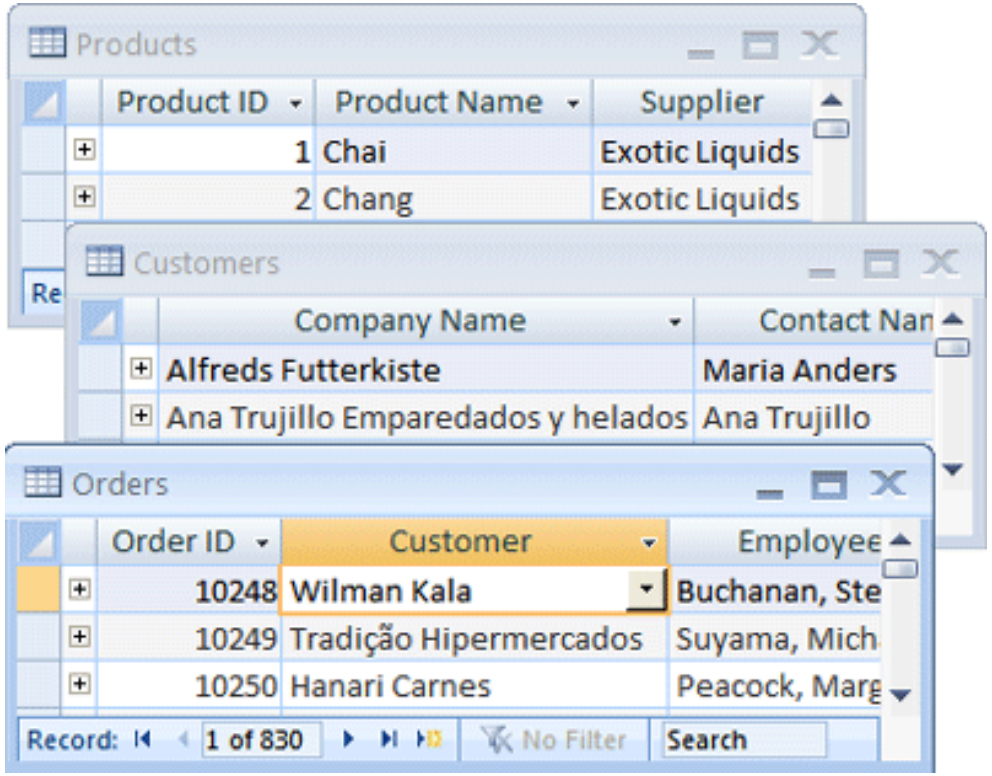
Unit-4
Designing Databases

Designing Databases

Access organizes your information into **tables**: lists of rows and columns reminiscent of an accountant's pad or a spreadsheet. In a simple database, you might have only one table. For most databases you will need more than one. For example, you might have a table that stores information about products, another table that stores information about orders and another table with information about customers.

Designing Databases



Designing Databases

Each row is more correctly called a **record** and each column, a **field**. A record is a meaningful and consistent way to combine information about something. A field is a single item of information — an item type that appears in every record. In the Products table, for instance, each row or record would hold information about one product. Each column or field holds some type of information about that product, such as its name or price.

Database Design

The design process consists of the following steps:

- Determine the purpose of your database.
- Find and organize the information required.
- Divide the information into tables.
- Turn information items into columns.
- Specify primary keys.
- Set up the table relationships.
- Refine your design.
- Apply the normalization rules.

The design process

The design process consists of the following steps:

Determine the purpose of your database

This helps prepare you for the remaining steps.

Find and organize the information required

Gather all of the types of information you might want to record in the database, such as product name and order number.

Divide the information into tables

Divide your information items into major entities or subjects, such as Products or Orders. Each subject then becomes a table.

The design process

Turn information items into columns

Decide what information you want to store in each table. Each item becomes a field, and is displayed as a column in the table. For example, an Employees table might include fields such as Last Name and Hire Date.

Specify primary keys

Choose each table's primary key. The primary key is a column that is used to uniquely identify each row. An example might be Product ID or Order ID.

Set up the table relationships

Look at each table and decide how the data in one table is related to the data in other tables. Add fields to tables or create new tables to clarify the relationships, as necessary.

The design process

Refine your design

Analyze your design for errors. Create the tables and add a few records of sample data. See if you can get the results you want from your tables. Make adjustments to the design, as needed.

Apply the normalization rules

Apply the data normalization rules to see if your tables are structured correctly. Make adjustments to the tables, as needed.

Relational Database Model

Relational Model (RM) represents the database as a collection of relations. A relation is nothing but a table of values. Every row in the table represents a collection of related data values. These rows in the table denote a real-world entity or relationship.

The table name and column names are helpful to interpret the meaning of values in each row. The data are represented as a set of relations. In the relational model, data are stored as tables. However, the physical storage of the data is independent of the way the data are logically organized.

Relational Database Model

Some popular Relational Database management systems are:

- DB2 and Informix Dynamic Server – IBM
- Oracle and RDB – Oracle
- SQL Server and Access – Microsoft

Relational Model Concepts in DBMS

Attribute: Each column in a Table. Attributes are the properties which define a relation. e.g., Student-id, Roll no, NAME etc.

Tables – In the Relational model the, relations are saved in the table format. It is stored along with its entities. A table has two properties rows and columns. Rows represent records and columns represent attributes.

Tuple – It is nothing but a single row of a table, which contains a single record.

Relation Schema: A relation schema represents the name of the relation with its attributes.

Relational Database Model

Degree: The total number of attributes which in the relation is called the degree of the relation.

Cardinality: Total number of rows present in the Table.

Column: The column represents the set of values for a specific attribute.

Relation instance – Relation instance is a finite set of tuples in the RDBMS system. Relation instances never have duplicate tuples.

Relation key – Every row has one, two or multiple attributes, which is called relation key.

Attribute domain – Every attribute has some pre-defined value and scope which is known as attribute domain.

Relational Database Model

Table also called Relation

Primary Key

Domain
Ex: NOT NULL

© guru99.com

CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive

Tuple OR Row

Total # of rows is Cardinality

Column OR Attributes

Total # of column is Degree

Relational Integrity Constraints

Relational Integrity constraints in DBMS are referred to conditions which must be present for a valid relation. These Relational constraints in DBMS are derived from the rules in the mini-world that the database represents.

There are many types of Integrity Constraints in DBMS. Constraints on the Relational database management system is mostly divided into three main categories are:

- Domain Constraints
- Key Constraints
- Referential Integrity Constraints

Domain Constraints

Domain constraints can be violated if an attribute value is not appearing in the corresponding domain or it is not of the appropriate data type.

Domain constraints specify that within each tuple and the value of each attribute must be unique. This is specified as data types which include standard data types integers, real numbers, characters, Booleans, variable length strings, etc.

Example:

Create DOMAIN CustomerName

CHECK (value not NULL)

The example shown demonstrates creating a domain constraint such that CustomerName is not NULL

Key Constraints

An attribute that can uniquely identify a tuple in a relation is called the key of the table. The value of the attribute for different tuples in the relation has to be unique.

Example

In the given table, Customer ID is a key attribute of Customer Table. It is most likely to have a single key for one customer, Customer ID =1 is only for the Customer Name =” Google”.

Example:

Customer ID	Customer Name	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive

Normalization

Normalization is the process of organizing data in a database. It includes creating tables and establishing relationships between those tables according to rules designed both to protect the data and to make the database more flexible by eliminating redundancy and inconsistent dependency.

The main objective of database normalization is to eliminate redundancy, minimize data modification errors, and simplify the query process.

Why normalization can be needed?

The main objective of database normalization is to eliminate redundant data, minimize data modification errors and simplify the query process. Ultimately, normalization goes beyond simply standardizing data and can even improve workflow, increase security and lessen costs.

Types of Normalization

1. First Normal Form(1NF)
2. Second Normal Form(2NF)
3. Third Normal Form(3NF)

Types of Normalization

1. First Normal Form(1NF)

A Table is said to be in 1NF if:

- All columns contain atomic values.
- All values of a column(attribute) belong to the same domain.
- Column names are unique.

For example, if there are multiple branch names for a student, they must be converted to a single-valued attribute as shown below.

Types of Normalization

Conversion of Table into First Normal Form (1NF)

ID	Name	Branch Name
1.	John	CS, Civil
2.	Ben	Electronics
3.	Steve	Mechanical
4.		
5.		

ID	Name	Branch Name
1.	John	CS
2.	John	Civil
3.	Ben	Electronics
4.	Steve	Mechanical
5.		

Types of Normalization

2. Second Normal Form (2NF)

A Table (Relation) in the Database exists in the Second Normal Form if:

- It exists in the 1NF.
- It does not have Partial Dependency

Let’s understand the Second Normal Form of Normalization in DBMS with an example. Suppose there is a table ‘Product’ containing the Customer ID, Name, and Price of Products.

Types of Normalization

Customer_ID	Name	Price
1	Fan	1000
2	AC	1500
1	Phone	2000
4	Lamp	1000
4	Fan	1000
2	Earphones	2000

Types of Normalization

Here,

Candidate Keys are {Customer_ID, Name}. The Prime Attributes (attributes that are a part of the candidate key) will be Customer_ID and Name and the Non-Prime Attribute will be Price. We see that Customer_ID->Price is a Partial Functional Dependency because Price (non-prime attribute) depends on Customer_ID(a subset of the candidate key).

Types of Normalization

Conversion to Second Normal Form (2NF)

Table 1

Customer ID	Product Name
1	Fan
1	Phone
2	AC
2	Earphones
4	Lamp
4	Fan

Table 2

Product Name	Price
Fan	1000
Phone	2000
AC	1500
Earphones	2000
Lamp	1000

Types of Normalization

Both the above tables are in 1NF. Also, the Functional Dependencies are Customer ID->Product Name and Product Name->Price in which there is no Partial Dependency. Also, the price of the Fan is not stored multiple times. So, Data Redundancy is removed.

Types of Normalization

3. Third Normal Form (3NF)

The Third Normal Form exists in a table if:

It exists in the 2NF.

There is no Transitive Dependency for non-prime attributes in the table.

Now, let’s first see Transitive Functional Dependency before going ahead with the Normalization in DBMS.

Types of Normalization

Transitive Functional Dependency

Transitive Functional Dependency occurs when a non-prime attribute is dependent on another non-prime attribute. For example, in a relation, if Student ID determines Student’s City then Student ID -> Student City is a Functional Dependency. And, if Student’s State is determined by Student’s City then Student City->Student State is also a Functional Dependency.

Types of Normalization

STU_ID	STU_NAME	ZIPCODE	STU_STATE	STU_CITY
1.	Harry	201010	UP	Noida
2.	Stephan	02228	US	Boston
3.	Lan	60007	US	Chicago
4.	Katharine	06389	UK	Norwich
5.	John	462007	MP	Bhopal

Types of Normalization

In this table, the Candidate Key is Student ID which is a prime attribute, and the rest of the attributes are Non-Prime. Thus, there is no partial dependency. So, it is in 2NF.

But, STU_ID->ZIPCODE and ZIPCODE->STU_STATE are the two functional dependencies that exist. Since Zipcode is a non-prime attribute, STU_STATE indirectly depends on STU_ID. Therefore, STU_ID->STU_STATE also exists which is a Transitive Functional Dependency.

To convert this into 3NF, we decompose the relation into two tables as below.

Types of Normalization

Conversion to Third Normal Form (3NF)

Table 1

STU_ID	STU_NAME	ZIPCODE
1.	Harry	201010
2.	Stephan	02228
3.	Lan	60007
4.	Katharine	06389
5.	John	462007

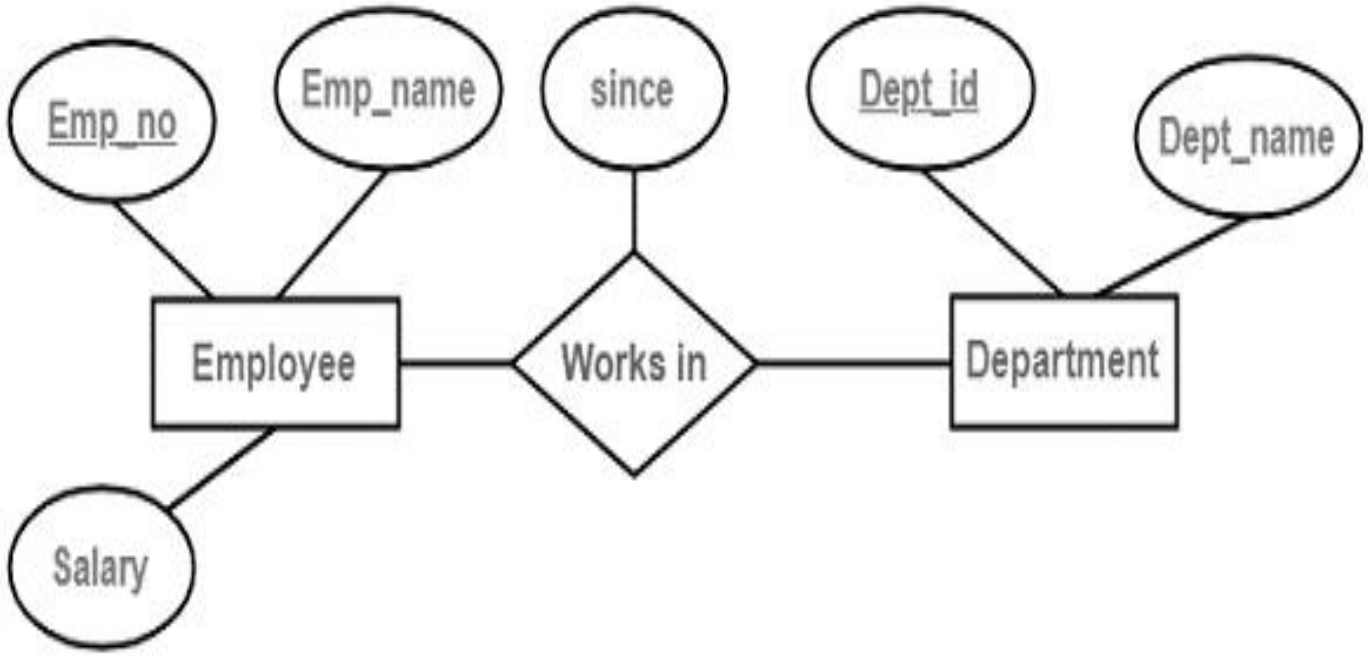
Table 2

ZIPCODE	STU_STATE	STU_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich
462007	MP	Bhopal

Types of Normalization

Now, both the tables are free from Transitive Functional Dependency and thus exist in 3NF of Normalization in DBMS.

E-R diagram of Employee Management System



Transforming E-R diagram into Relations

Emp_no	Dept_id	since

Schema : Works in (Emp_no , Dept_id , since)

Merging Relations

For example I have created a relational schema (for an online bookstore) after decomposing relations and getting them to 3NF. Here are two of the relations.
ShoppingCart(CartID{PK}, TimeStamp) CustomerCart(CartID{PK and FK}, Email)
Can these merge? Is the only rule to merge these two together that they need to have the same primary key? What about CartID in customer cart also being a foreign key?

They can't have the same primary key otherwise it wouldn't be a primary key. You can merge the two if the primary key of one table is included somewhere in the other (foreign key).

Physical File

A *physical file* is a description of how data is to be presented to or received from a program and how data is actually stored in the database. A physical file contains one record format and one or more members. You can create and describe a physical file using the traditional system interface.

- **Creating a physical file**
  You can create a physical file using data description specifications (DDS).
- **Specifying physical file and member attributes**
  You can specify the attributes of physical files and members on the Create Physical File (CRTPF), Create Source Physical File (CRTSRCPF), Change Physical File (CHGPF), Change Source Physical File (CHGSRCPF), Add Physical File Member (ADDPFM), and Change Physical File Member (CHGPFM) commands.
- **Implicit physical file journaling**
  A physical file can be journaled automatically when it is created.

Database Design

Following are some critical points to keep in mind to achieve a good database design:

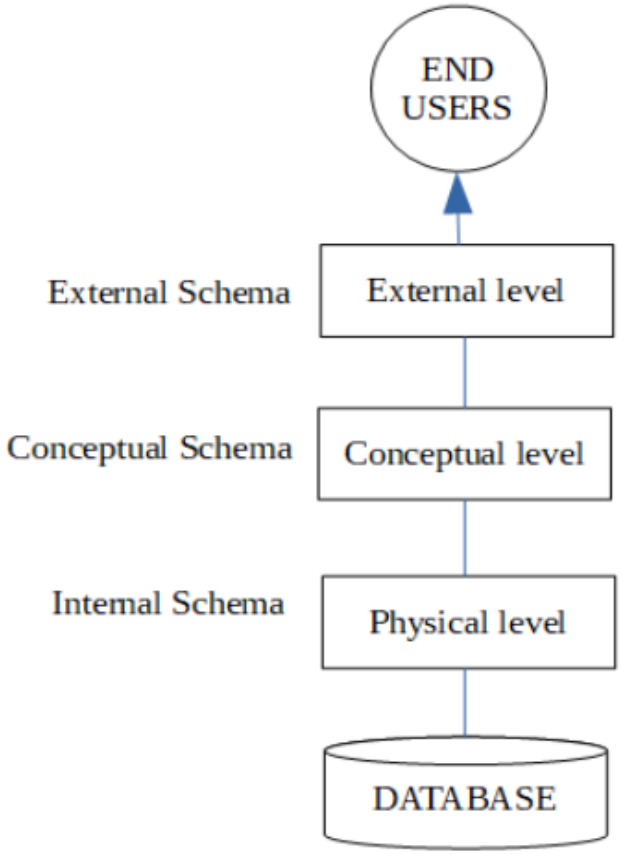
- Data consistency and integrity must be maintained.
- Low Redundancy
- Faster searching through indices
- Security measures should be taken by enforcing various integrity constraints.
- Data should be stored in fragmented bits of information in the most atomic format possible.

Database Design

Database design or database levels are

- Conceptual database design(External level)
- Logical database design(Internal level)
- Physical database design

Database Design



Designing Fields

A field is the part of an application that collects data. You create fields on forms, sub forms, or in layout regions. Each field stores a single type of information. A field's field type defines the kind of information a field accepts, such as text, numbers, dates, or names. When a user, either in a Notes client or a Web browser, creates a form, fills out the information in the fields, and saves the form, the data in the fields is stored in an individual document. The contents of the fields can then be displayed in documents and views or can be retrieved for use in formulas. A field can be used on a single form or you can create shared fields for use in multiple forms in a database.

Designing Fields

You can create a single-use field on a single form or a shared field to use on several forms. In either case, you must specify the following:

- Field name
- Field type

Whether this field is editable or computed.

Designing Physical Tables

During physical design, you transform the entities into tables, the instances into rows and the attributes into columns. You and your colleagues must decide on many factors that affect the physical design such as:

- How to translate entities into physical tables
- What attributes to use for columns of the physical tables
- Which columns of the tables to define as keys
- What indexes to define on the tables
- What views to define on the tables
- How to de normalize the tables